

박재홍

Java와 Spring을 기반으로 업무 규칙을 데이터 구조와 API로 구현하고, 테스트와 배포 절차를 통해 운영을 고려한 서비스로 구현합니다.

DATA INTEGRITY

트랜잭션 · DB 제약조건
상태 이력

CONCURRENCY & RETRY

Idempotency-Key · UNIQUE
409 오류 계약

DELIVERY & OPERATIONS

GCP · AWS · Health Check
Runbook

Backend / Platform Engineering

REST API와 도메인 로직, PostgreSQL 정합성, 동시 요청 제어, GCP·AWS 배포 자동화, 운영 Health Check와 문서화를 하나의 개발 범위로 다뤘습니다.

Core Competencies

Backend

Java 21, Spring Boot, Spring MVC, Spring Security, JPA

Data

PostgreSQL, PostGIS, Redis, Flyway, DB constraint

Delivery

GitHub Actions, GCP VM, AWS EC2, Docker, GHCR

Quality

JUnit 5, MockMvc, Testcontainers, JaCoCo, Runbook

Representative Projects

위굴 Backend

BACKEND · GCP QA DELIVERY

예산, 위시, 구매 결정, 알림, 회고가 연결되는 소비 의사결정 서비스입니다.

- 도메인 스키마 및 상태 전이 설계
- PostgreSQL 제약조건과 Testcontainers 검증
- Idempotency-Key 기반 재시도 안전성
- GCP VM standby 배포와 운영 Runbook

Spring Boot

PostgreSQL

Flyway

GCP

Beach Complex

BACKEND · INFRA

위치 기반 해수욕장 정보, 상태 수집, 예약과 알림을 제공하는 서비스입니다.

- 혼잡도·날씨 외부 데이터 주기 수집
- 예약 API와 동시 요청 통합 테스트
- AWS EC2·GHCR·Docker Compose CD
- Firebase 런타임 구성과 환경 분리

PostGIS

Redis

AWS EC2

Docker

Engineering Profile

상태와 이력

현재 상태와 변경 이력을 분리하고, 여러 테이블의 갱신을 하나의 트랜잭션으로 처리했습니다.

재시도와 동시성

Idempotency-Key, UNIQUE 제약과 409 오류 계약으로 중복 요청을 제어했습니다.

배포와 운영

후보 버전 Health Check, 이미지 버전 추적, runtime secret과 배포 실패 시 상태·로그 확인을 자동화했습니다.

소비 의사결정 흐름을 하나의 백엔드 도메인으로 연결

사용자 인증부터 상품 저장, 구매 속려, 예산 반영, 리마인더와 회고까지 이어지는 Spring Boot 기반 서비스

프로젝트 개요

구매를 고민하는 상품을 저장하고, 예산과 셀프체크를 바탕으로 구매 여부를 결정하도록 돕는 서비스입니다.

기여 영역

- 초기 Spring Boot·JPA·Flyway 기준선
- Budget, Wishlist, Decision, Notification 도메인
- JWT·OAuth 기반 인증 흐름
- GCP QA 배포와 FCM 런타임 설정
- CI 테스트·배포 후 Health Check 자동화

Client

REST API
JWT / OAuth
X-Request-Id

Spring Boot

Auth · Wishlist · Budget ·
Decision · Notification ·
Reflection · Social

PostgreSQL

JPA · Flyway
CHECK / UNIQUE
partial index · trigger

CI

GitHub Actions
Test · bootJar
JaCoCo

GCP QA

Caddy · systemd
8080/8081 standby

Operations

Health Check
Runtime Secret
Runbook

Service Flow

1 인증 OAuth / JWT 사용자 식별	2 상품 저장 직접 입력 또는 URL 위시 상태 생성	3 구매 속려 예산과 소비 이력 셀프체크	4 GO / STOP 구매 결정과 상태 전이	5 리마인더 7일 후 회고 알림 예약	6 회고 구매 후 결과와 소비 이력
--	--	---	---	---	--

Domain Model

도메인	주요 데이터	설계 포인트
Budget	월 예산, 사용 금액, 경고 기준	사용자·월 단위 유일성, 음수 금액 차단, 결정 결과와 지출의 일관성
Wishlist	상품 상태, 가격, 입력 출처, idempotency key	URL 중복과 요청 재시도 중복을 서로 다른 규칙으로 처리
Decision	GO/STOP, 셀프체크, 합리성 결과, 변경 이력	여러 테이블의 상태를 단일 트랜잭션으로 갱신하고 결정 당시 결과를 보존
Notification	리마인더, 인앱 알림, 디바이스 토큰	예약 시각·취소 상태·발송 상태의 정합성과 FCM runtime secret 분리

Technical Decisions

PostgreSQL 기준선

H2 대신 Testcontainers PostgreSQL로 JSONB, partial index, trigger와 실제 Flyway migration을 검증했습니다.

상태 일관성

서비스 검증과 DB 제약을 함께 사용해 잘못된 합리성 결과, 음수 금액과 활성 중복 데이터를 차단했습니다.

과거 결과 보존

결과 재조회 시 최신 캐릭터 상태가 아니라 구매 결정 당시 이벤트를 반환하도록 설계했습니다.

Key API Contracts

Wishlist

POST /api/v1/wishlist/items
상품 저장과 중복 정책

Decision

POST /api/v1/decisions
GO/STOP 상태 전이

Result

GET
/api/v1/decisions/{id}/result
결정 당시 결과 조회

Push

POST /api/v1/push-tokens
디바이스 토큰 등록

정합성, 재시도 제어와 배포 실패 방어를 코드와 절차로 구현

1. 구매 결정 트랜잭션과 데이터 정합성

1 검증 사용자·상품 ·답변	2 결정 GO / STOP 저장	3 상태 saved_items 변경	4 예산 GO일 때 지 출 증가	5 알림 7일 리마인더	6 이벤트 결정 당시 반 응
---------------------------------	-----------------------------------	-------------------------------------	-----------------------------------	---------------------------	---------------------------------

구매 결정 관련 변경을 하나의 트랜잭션으로 처리해 일부 상태만 반영되는 경우를 방지했습니다.

DB 불변조건

- 금액과 횟수의 음수 저장 차단
- 셀프체크 Yes 개수 0~4 제한
- Yes 개수와 합리성 결과 일치
- 활성 데이터에만 partial unique index
- 투표 집계와 결정 결과 trigger 검증

2. 업무 데이터와 동일 요청의 재전송 분리

문제 구분

같은 상품 내용

서로 다른 구매 고민일 수 있으므로 저장을 허용합니다.

같은 생성 요청의 재전송

더블탭이나 timeout 재시도로 중복 row가 생기지 않아야 합니다.

```
alter table saved_items
  add column idempotency_key varchar(120);

create unique index uk_saved_items_user_idempotency_key
  on saved_items(user_id, idempotency_key)
  where idempotency_key is not null;
```

Same key

기존 생성 결과 반환

No key

독립 항목 생성

URL item

기존 URL 중복 유지

코드 리뷰에서 재시도 방어 공백을 확인한 뒤, content 기반 dedupe를 되살리지 않고 Idempotency-Key 계약과 DB partial unique index로 보완했습니다.

3. GCP QA 배포 자동화

1 Build Java 21 test + bootJar	2 Candidate SHA 기반 JAR VM 업로드	3 Standby 8081 선기동 Health Check	4 Switch Caddy를 8081로 트래픽 전환	5 Recover 8080 교체·확인 후 Caddy 복귀
--	---	---	--	---

배포 전 검증

- prod profile과 JWT secret 검증
- trusted header와 local redirect 차단
- Caddy, systemd, PostgreSQL 상태 확인
- 8081 포트 점유 여부 확인

운영 연결

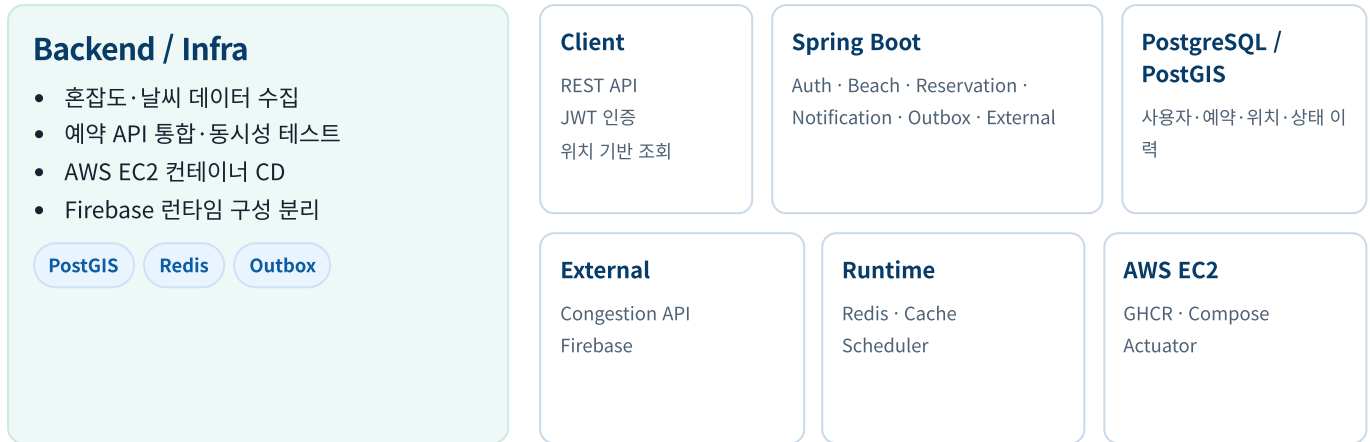
- FCM 자격증명은 VM-local 파일로 관리
- systemd drop-in으로 runtime secret 주입
- X-Request-Id로 요청과 journalctl 로그 추적
- 8080 복구 실패 시 8081이 트래픽을 처리하도록 구성

검증 지점	성공 기준	실패 처리
Candidate	8081 /health 통과 후에만 Caddy 전환	전환 전 실패 시 후보 프로세스 종료
Caddy	설정 format-validate 후 reload	백업 Caddyfile 복원
Main	8080 /health와 외부 HTTPS 경로 확인	복구 전까지 8081 트래픽 유지
Environment	prod profile, secret 길이, redirect, reviewer seed 확인	배포 시작 전 fail-fast

GitHub: [PR #20](#) · [PR #180](#) · [PR #174](#)

외부 상태 데이터, 위치 기반 정보, 예약과 알림을 백엔드 서비스로 구성

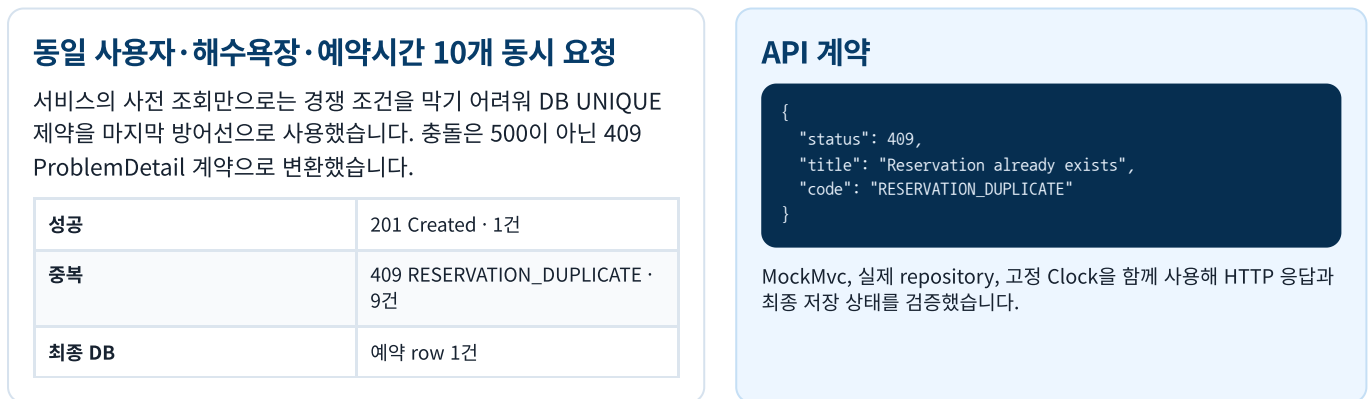
공공데이터와 외부 혼잡도 정보를 수집하고, 예약 API와 AWS 배포를 구현한 Spring Boot 프로젝트



상태 데이터 수집과 현재 상태 갱신



예약 API 동시성 검증



검증 영역	테스트 방법	확인 결과
시간 경계	고정 Clock으로 now-past-future 입력	동일 실행마다 결정적인 응답
소유권	다른 사용자의 예약 취소 요청	리소스 노출 없이 404 반환
동시성	10 thread에서 동일 예약 생성	201 1건, 409 9건, DB row 1건

GitHub: [PR #96](#) · [PR #155](#)

AWS 배포 자동화와 테스트·운영 문서 체계 구축

AWS EC2 Backend CD

1 Build Docker Buildx 애플리케이션 이미지	2 Publish GHCR latest와 commit SHA 태그	3 Connect GitHub Actions에서 EC2 SSH 접속	4 Deploy Compose pull/up runtime env 주입	5 Verify Actuator Health 실패 시 ps-logs 출력
---	---	--	--	---

이미지 버전과 배포 결과 추적

PR #209

- latest는 현재 배포 기준, commit SHA는 특정 버전 추적에 사용했습니다.
- GitHub Secrets와 EC2 env 파일을 분리해 비밀값을 이미지에 포함하지 않았습니다.
- Health Check 실패 시 컨테이너 상태와 최근 로그를 즉시 출력하도록 구성했습니다.

외부 연동의 런타임 구성 분리

PR #216

- FirebaseMessaging 빈이 존재할 때만 Outbox 발송 체인을 등록했습니다.
- 비활성 환경에서는 앱이 정상 기동하고, 활성 환경에서는 외부 secret을 요구합니다.
- 조건부 빈 등록을 configuration 레이어에 모아 서비스 코드의 환경 분기를 제거했습니다.

Testing Strategy

검증 레이어	도구	검증 대상
Unit	JUnit 5 · Mockito	계산, 분기, 토큰 정책, 외부 전송 위임과 예외 전파
API Integration	MockMvc · Spring Context	인증, 입력 경계, HTTP 상태, ProblemDetail 오류 계약
Database	Testcontainers PostgreSQL	Flyway, JSONB, constraint, partial index, trigger와 최종 row 상태
Delivery	Actuator · curl · workflow	후보 애플리케이션 기동, 배포 후 Health, 컨테이너 로그

18 tests

이메일 전송 · 비동기 조립 · 토큰 정책 단위 테스트

97.0%

대상 5개 클래스 Line coverage

92.9%

대상 5개 클래스 Branch coverage

프레임워크 경계인 AFTER_COMMIT과 @Retryable 실행작은 단위 테스트에서 분리해 통합 테스트 대상으로 남겼습니다.

Documentation & Operations

개발 기준 문서

마일스톤, 핵심 API, 레이어 책임, 입력 검증과 Definition of Done

ADR

Testcontainers, 고정 Clock, API 오류 계약 등 기술 선택과 대안

Runbook

Health Check, Request ID, journalctl, runtime secret과 재시작 절차

CI Quality

Gradle test, JaCoCo, Checkstyle, Spotless와 Docker build

Runtime Secret

Firebase key와 운영 env를 이미지·저장소 밖에서 주입

Health

Actuator와 curl 재시도로 배포 후 기동 상태 확인

Error Contract

ProblemDetail의 status·title·code를 통합 테스트로 고정

Reviewability

PR에 범위, 테스트, 리스크와 롤백 기준을 함께 기록

GitHub: PR #209 · PR #216 · PR #186

정합성·재시도 제어·배포 실패 방어를 코드와 검증 절차로 구현

두 프로젝트에서 적용하고 검증한 백엔드 엔지니어링 원칙

Engineering Cases

01	DATA INTEGRITY	IMPLEMENTATION	EVIDENCE
데이터 정합성	구매 결정 과정의 예산·위시 상태·알림·이벤트 변경을 하나의 트랜잭션으로 처리했습니다. CHECK·UNIQUE·partial index·trigger로 서비스 규칙을 PostgreSQL 불변조건으로 확장했습니다.	구매 결정 과정의 예산·위시 상태·알림·이벤트 변경을 하나의 트랜잭션으로 처리했습니다. CHECK·UNIQUE·partial index·trigger로 서비스 규칙을 PostgreSQL 불변조건으로 확장했습니다.	Wigul PR #6 · #20 Testcontainers PostgreSQL Flyway · constraint · trigger
02	CONCURRENCY & RETRY	IMPLEMENTATION	EVIDENCE
재시도와 동시 요청 제어	동일 요청 재전송은 Idempotency-Key와 partial unique index로 제어했습니다. 예약 API는 동일 조건의 10개 동시 요청에서 201 1건, 409 9건, 최종 DB row 1건을 검증했습니다.	동일 요청 재전송은 Idempotency-Key와 partial unique index로 제어했습니다. 예약 API는 동일 조건의 10개 동시 요청에서 201 1건, 409 9건, 최종 DB row 1건을 검증했습니다.	Wigul PR #180 Beach Complex PR #155 201 × 1 · 409 × 9 · Row × 1
03	DELIVERY & OPERATIONS	IMPLEMENTATION	EVIDENCE
배포 검증과 실패 대응	GCP에서는 후보 애플리케이션의 Health Check 통과 후에만 트래픽을 전환하고, 전환 전 실패 시 기존 서비스를 유지하도록 구성했습니다. AWS에서는 commit SHA, runtime secret 분리와 배포 후 상태·로그 출력을 통해 배포 버전과 실패 원인을 추적할 수 있도록 구성했습니다.	GCP에서는 후보 애플리케이션의 Health Check 통과 후에만 트래픽을 전환하고, 전환 전 실패 시 기존 서비스를 유지하도록 구성했습니다. AWS에서는 commit SHA, runtime secret 분리와 배포 후 상태·로그 출력을 통해 배포 버전과 실패 원인을 추적할 수 있도록 구성했습니다.	Wigul PR #174 Beach PR #209 · #216 Candidate · health · SHA trace

Verification Snapshot

CONCURRENCY TEST	DATABASE TEST	DELIVERY CHECK
10 REQUESTS	POSTGRESQL	CANDIDATE / HEALTH
201 × 1 · 409 × 9 Final DB row × 1	Flyway · JSONB · Constraint Partial index · Trigger	Traffic switch after pass Failure logs · SHA trace

Project Evidence

위굴 Backend github.com/SaGongSa-404/404_BE PR #6 도메인·스키마 기준선 PR #20 구매 결정 트랜잭션 PR #174 GCP candidate 배포 PR #180 Idempotency-Key	Beach Complex github.com/Beach-complex/Beach_complex PR #96 외부 상태 수집 PR #155 예약 동시성 검증 PR #209 AWS EC2 CD PR #216 외부 연동 runtime 구성
---	---

박재홍 · Backend Engineer
 업무 규칙을 명확한 데이터 구조와 API로 구현하고, 실제 데이터베이스 검증과 배포 절차를 통해 운영을 고려한 서비스로 구현합니다.

github.com/PHJ2000